So, at height h, we have 2^h nodes. But we also know that the total number of nodes for a complete tree with height h is $2^0 + 2^1 + 2^2 + 2^3 + \ldots + 2h = 2^{h+1} - 1$.

To keep the height as small as possible, it is easy to see that we need to fill all the levels (except, possibly, the last) because otherwise, we could just move the nodes in the last level up the tree until we have no vacancies in upper levels.

Therefore, if we have n nodes, $2^{h+1} - 1 \leq n$, and taking the log of both sides, $h \geq \log(n+1) - 1$.

*BST*s are not balanced by nature: on the contrary, for the same set of elements, their shape and height can greatly vary depending on the sequence of insertions for those elements (see figure C.10).

On average, after a high number of insertions, the probability to have a skewed tree is pretty low. It is worth mentioning, however, that the simplest algorithm for deleting a node tends to produce skewed trees. Many workarounds have been proposed for this problem but, as of today, there is no proof that they always produce a better result than the naive version.

The silver lining is that there are several solutions that allows to keep *BST*s balanced without degrading performance for insert or delete. For instance:

- 2-3 search trees
- red-black trees
- B-trees
- AVL trees

C.2 Hash Table

Hashing is probably the most common way to represent symbol tables. If we need to associate keys, taken from a (multi)set, each one to a value, we are going to face a few problems.